EXECUTE THE PROGRAM: AUDITING MESH-TO-CAD RECONSTRUCTION AND AGENT TOOLS

Anonymous authors
Paper under double-blind review

ABSTRACT

Optimization-based mesh-to-CAD reconstructors are steered by the geometry their search scores, but they deliver a program, and that program can fail to build or build an invalid solid. We score systems by executing the delivered program in a sandbox, requiring every solid to be OCC-valid, and comparing the executed solid with the target, with every failed delivery scored zero. On 300 held-out hard-tier CADBench parts, a beam-search configuration of the CADFit optimizer obtains no valid replay on 41 parts; delivering the best candidate program that executes to a valid solid raises validity-gated IoU by +0.1037 (95% interval [+0.0709, +0.1376]), the preregistered primary contrast, measured under a dated recovery amendment for replay failures and reported beside the original measurement. Under the same judge, Claude Opus 5.5 writing CadQuery through a harness that executes every change scores 0.975 on 40 reserved held-out parts, +0.024 above the improved optimizer (exploratory). The optimizer's stages, offered to the agent as tools, did not raise its score in their original implementation; a revised package of output-preserving rewrites and a per-check time cap, evaluated on 40 new parts, halves Opus's mean time with these tools (8.9 to 4.2 minutes per part) and makes them beat plain code by +0.0032 (Holm $p = 0.028$), while its preregistered quality gain over the original package (+0.0367) is not significant by its declared sign test.

1 INTRODUCTION

Optimization-based mesh-to-CAD reconstructors such as CADFit (Nehme et al., 2026) keep the composition of fitted extrusions and revolves with the highest volumetric IoU, computed on meshes, and write out a CAD program at the end. The search's score does not say whether that program builds the scored geometry, yet the program is the product: a designer edits its sketches and features, and a CAD kernel turns it into a solid. Agentic systems face the same gap whenever an agent's tools return geometry rather than a checked program.

We close the gap at evaluation time. Each system's delivered program is executed in a sandbox, every solid it builds must be OCC-valid, the executed solid is scored with a judge that does not depend on how clean its tessellation is, and any failed delivery scores zero with every part kept in the denominator (Section 3). Each study fixes its arms, cohort and analysis before it runs; cohorts exclude every model index used before; every amendment and rerun is recorded with its date and cause.

Applied to a reconstruction optimizer and to frontier language-model agents, this protocol gives three findings (Figure 1).

1. **Execution-verified delivery is the largest single improvement to the optimizer** (Section 4). A beam-search configuration of CADFit (d0) obtains no valid replay on 41 of 300 held-out parts: 23 programs build an OCC-invalid solid, 7 raise a build error, 1 is missing and 10 exceed the replay limits. Executing each candidate program and delivering the best valid one, with a fuse-and-repair fallback, raises validity-gated IoU by +0.1037 [+0.0709, +0.1376] (preregistered primary). Geometry fixes, precise extrusion extents and a silhouette trim each add a smaller, significant increment.

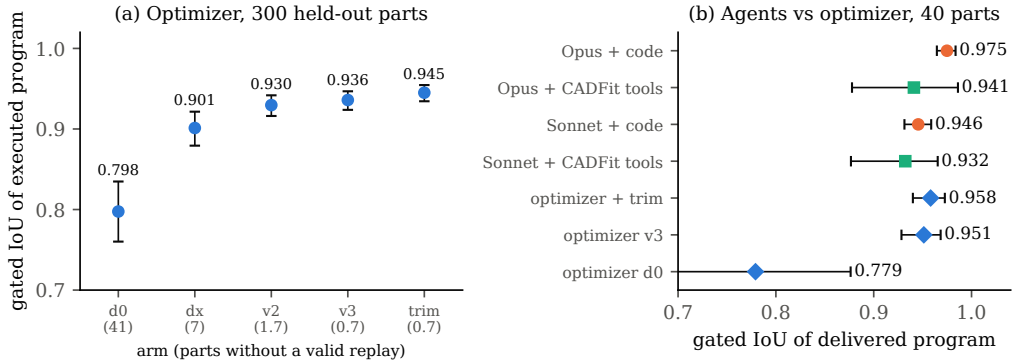


Figure 1: **(a)** The optimizer on 300 held-out fresh2 parts: validity-gated IoU of the executed program for the beam-search baseline d0 and each added stage (dots: mean over parts, three-seed arms averaged within part; bars: 95% split-stratified bootstrap intervals), with the number of parts without a valid replay (mean over seeds). **(b)** The 40 reserved fresh2 parts: Claude agents (three episodes per part, each delivered program re-judged) and the optimizer arms on the same parts.

2. **A frontier agent with plain code exceeds the optimizer's v3 on reserved held-out parts** (Section 5). Claude Opus 5.5 writing CadQuery through a harness that executes and checks every change scores 0.975, with 120/120 valid deliveries, against 0.951 for v3 (difference +0.024 [+0.003, +0.048]; exploratory); against the optimizer with trim the difference, +0.017 [−0.001, +0.036], is not established. With the optimizer's stages as additional tools, the agent's mean score does not rise, and 5 of 120 episodes end at the deadline with nothing delivered.
3. **A revised tool package makes the optimizer's stages pay off for the agent** (Section 6). Profiling located most of the tools' time in redundant computation. A package of output-preserving rewrites plus a 120 s cap per executed check, evaluated on a cohort drawn after it was frozen, halves Opus's time with the optimizer's tools and leaves every Opus delivery valid; with it those tools beat plain code by +0.0032 (Holm $p = 0.028$). Its preregistered quality gain over the original tools, +0.0367 [+0.0018, +0.0957], is not significant by the declared sign test ($p = 0.121$).

2 Related work

Mesh- and point-cloud-to-CAD. Learned methods map geometry to construction sequences: DeepCAD (Wu et al., 2021) and Point2Cyl (Uy et al., 2022) predict sketch-and-extrude structure, zone graphs (Xu et al., 2021) search over extrusion sequences, and CAD-Recode (Rukhovich et al., 2025) decodes point clouds into CadQuery code that is executed for evaluation. CADFit (Nehme et al., 2026) optimizes instead: it fits extrusion and revolve candidates to slices of the mesh and composes them greedily to maximize volumetric IoU. Datasets include ABC (Koch et al., 2019), Fusion 360 Gallery (Willis et al., 2021) and DeepCAD; CADBench (Doris et al., 2026) unifies them with face-count tiers and metrics for fidelity, executability and compactness. Our contribution is an audit-and-repair study of one optimizer under an executed, validity-gated endpoint, with each repair tested on held-out parts.

Iterative and agentic CAD generation. Tool-using language models (Yao et al., 2023; Schick et al., 2023) and agent–computer interfaces (Yang et al., 2024) motivate CAD agents that act, observe and revise. CADReasoner (Kabisov et al., 2026) trains a model to edit a runnable CadQuery program from the geometric discrepancy of its previous output, and IterCAD (Hu et al., 2026) trains a multimodal agent against an executable CAD sandbox. CAD-Assistant (Mallis et al., 2025) plans FreeCAD actions with CAD-specific tools, and CADFather (Savrasov et al., 2026) coordinates learned and algorithmic proposal tools with a vision-language assistant, executes the proposed programs and keeps the best valid one. CADSmith (Barkley et al., 2026) wraps text-to-CadQuery

Table 1: Positioning by evaluation design. *Search signal*: what the system optimizes or selects on. *Final replay*: the reported score is computed on the solid built by re-executing the final delivered program. *Zero-inclusive*: failed deliveries enter the fidelity average as zero. *Prospective*: arms and analysis fixed before a cohort drawn for the study. n.r.: not reported in that work.

Work	Input	Search signal	Final replay	Zero-inclusive	Prospective
CADFit	mesh	mesh IoU; kernel validity reported	n.r.	n.r.	n.r.
CAD-Recode	point cloud	learned decoder	yes	n.r.	n.r.
CADReasoner	renders, points	executed output	yes	n.r.	n.r.
CADFather	mesh	executed programs	yes	n.r.	n.r.
This work	mesh	mesh IoU (optimizer), executed IoU (agents)	yes	yes	yes

generation in nested loops that fix execution errors and check geometry programmatically, and RA-CAD (Yan et al., 2026) learns to critique a program after executing it. We use frozen closed models without training, take the tools from one optimizer, and compare two tool packages on a cohort drawn after both were frozen.

3 EXECUTION-BASED EVALUATION

Executed program. Every system delivers a CadQuery program (CadQuery contributors, 2024). We run it in a fresh sandboxed process with no network access and a 900 s limit (the trim arm of Section 4 is the one exception, noted there), take its result solid, and check OCC validity of the shape and of every solid it contains. A program that fails to build, builds an invalid solid, exceeds the limit, or is missing is a failed delivery.

Judge. The executed solid is tessellated (linear tolerance 10^{-3}, angular 0.1) and compared with the target mesh, both normalized so that the target's largest extent spans $[-1, 1]$. When both tessellations are closed and consistently oriented, IoU is the exact IoU of the two polyhedra, computed with manifold booleans (Lambert & Manifold contributors, 2024) after unioning overlapping bodies; otherwise it is a seeded Monte-Carlo estimate in which inside-ness of the reconstruction is the generalized winding number (Jacobson et al., 2013), so a tessellation cracked by float round-off is not judged as a hollow shell (sample counts and stopping rule in Appendix F). **Validity-gated IoU** is the IoU of the executed solid if the delivery succeeded and 0 otherwise. Every cohort part stays in every denominator.

Cohorts. All parts are hard-tier CADBench (Doris et al., 2026) solids from two splits, benchA and benchE, drawn by seeded shuffles that exclude every part whose model index (the first field of its file id) appears in any earlier set. Development used separate sets (Appendix A). **fresh2** (150 + 150 parts) was drawn after development for the optimizer study; **40 reserved parts** (20 + 20) were drawn from fresh2's manifest before any fresh2 outcome existed and used for the agent study; **fresh3** (20 + 20) was drawn after the revised tools and their protocol were frozen. The optimizer's outcomes on fresh2's other 260 parts were seen before the agent studies began; fresh3 is the only cohort unseen by every analysis.

Statistics. All comparisons are paired over parts. Repeated runs or episodes are averaged within part first. Intervals are nominal 95% bootstrap intervals that resample parts within split (Efron & Tibshirani, 1993). Tests are exact two-sided sign tests on per-part differences; a part counts as better or worse when its difference exceeds ± 0.005 in the optimizer study and $\pm 10^{-9}$ in the agent studies, and as neutral otherwise. Holm correction (Holm, 1979) is applied within each declared family (stated per study). The interval and the sign test answer different questions: the interval bounds the mean difference, the sign test asks whether more parts improve than worsen, so a mean gain carried by a few parts can have an interval above zero and a non-significant sign test.

Preregistration and amendments. Each study's arms, cohort, primary and secondary comparisons and analysis code were committed before its runs. Changes made after outcomes existed are recorded as dated amendments, reported in the main text with their effect, and listed with every exclusion and rerun in Appendix D.

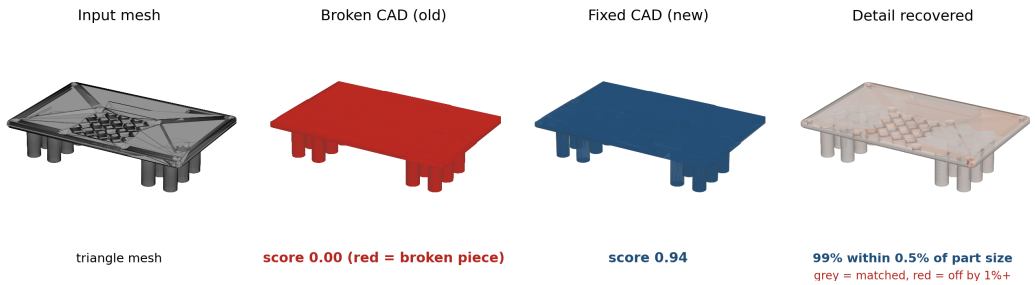


Figure 2: A development part (not from any evaluation cohort). Left to right: the input mesh; the program written by the optimizer's search, which stacks 208 overlapping solids of which 34 are OCC-invalid (red) and therefore scores 0; the program delivered by the execution check, a fused rewrite with one valid solid (IoU 0.94, as recorded by the pipeline); the input surface colored by distance to that delivery on a continuous scale from grey (0) to red (1% of the part size or more); 99% of the surface lies within 0.5%.

Table 2: Study 1, 300 held-out fresh2 parts (after the recovery amendment). Gated IoU of the executed program; v2, v3 and trim are averaged over three seeds within part and their valid counts are given per seed (min–max). Minutes: successful-attempt wall time per part (299 measured per run; three parts usually shared a node; retries excluded), mean over parts and median over parts; for three-seed arms both are averaged over the seeds' per-run values. Failure classes per run are in Table 5.

Arm	Gated IoU	benchA	benchE	Valid / 300	Mean min	Median min
d0 (baseline)	0.798	0.713	0.882	259	7.5	3.7
dx (+ exec.-verified delivery)	0.901	0.893	0.909	293	16.0	5.5
v2 (+ geometry fixes)	0.930	0.913	0.946	298–299	13.5	6.4
v3 (+ precise extents)	0.936	0.923	0.949	298–300	15.1	7.2
trim (+ silhouette trim)	0.945	0.933	0.957	299–300	partial (see text)	
section-only reference	[pending: section-only control (running)]					
Contrasts (difference, 95% interval, parts better/worse/neutral at ± 0.005, p)						
dx − d0 (primary)	+0.1037 [+0.0709, +0.1376], 43/4/253, $p < 10^{-4}$					
v2 − dx	+0.0284 [+0.0127, +0.0465], 57/20/223, Holm$_3$ $p < 10^{-4}$					
v3 − v2	+0.0063 [+0.0005, +0.0127], 69/38/193, Holm$_3$ p 0.004					
trim − v3	+0.0091 [+0.0048, +0.0134], 90/1/209, Holm$_3$ $p < 10^{-4}$					
trim − section-only	[pending: trim vs section-only (running)]					

4 Study 1: The optimizer on held-out parts

Arms. All arms run frozen source trees on the 300 fresh2 parts. **d0** is our pre-repair configuration of CADFit, frozen at one commit: on top of the released pipeline it scores compositions by exact mesh union, adds a cut phase, event-driven slicing and direction detection, runs a beam search of width 4, and proposes whole-part section extrusions; two infrastructure fixes common to every arm (a memory bound on mesh decimation with a retry of crashed CadQuery workers, and a seeded surface sampler for the revolve-axis search) are applied to it (exact flags in Appendix G). **dx** adds execution-verified delivery: the search's program and a rewrite of it that replaces stacked compounds by boolean fuse and cut are executed and judged, the better valid one is delivered, and if neither is valid the invalid solids are repaired (ShapeFix per solid, dropping a solid that stays invalid) before falling back to the search's program (Appendix G). **v2** builds section proposals from and scores the search against the undecimated input, and adds three geometry fixes (even-odd loop nesting, keeping loops down to 10^{-4} area, densifying revolve profiles at their split); **v3** adds precise extrusion extents (extent sweep, finer steps along thin directions, ends snapped to cap planes with the unsnapped candidate kept); **trim** adds a silhouette-trimmed variant (the best valid program intersected with prisms over the target's three axis silhouettes) as one more candidate for the execution check. d0 and dx run once; v2, v3 and trim run with three seeds, averaged within part. The primary contrast, dx − d0, is analysed alone; the secondary family (Holm over four tests) is v2 − dx, v3 − v2, trim − v3 and trim against a section-only reference (one executed mid-plane section extrusion per axis, best of three).

Execution-verified delivery. The baseline d0 obtains a valid replay on 259 of 300 parts. Of the 41 failures, 23 programs build an OCC-invalid solid, 7 raise a build error, 1 produced no program, and 10 exceed the replay limits (time or memory); the last class says nothing about the geometry and is scored 0 under the protocol. Delivering the best program that actually builds valid raises gated IoU from 0.798 to 0.901 (Table 2), a primary contrast of +0.1037 [+0.0709, +0.1376]. The gain comes from validity: on parts where both arms deliver a valid program the mean difference is +0.0035.

Later stages. Each later stage adds a smaller, significant increment: +0.0284 for the geometry fixes, +0.0063 for precise extents, and +0.0091 for the trim (better/worse 90/1). These adjusted p-values are Holm over the three available tests and are provisional until the fourth (trim against the section-only reference) is computed. The trim records are the trim stage's own replays, which use a 600 s limit instead of the common 900 s; a replay of the delivered trim programs under the common protocol is running ([pending: common-protocol replay of trim programs (running)]). The trim arm also re-runs the fuse/repair pass before trimming. Over all 300 parts and per seed, that pass changes gated IoU by −0.0031 to +0.0029 (the negative value is one seed-0 replay that timed out inside the trim run and is kept as a frozen 0), and the trim itself adds +0.0089 to +0.0096.

Recovery amendment. The first scoring pass stored two kinds of replay failure: on one node the sandbox could not be set up, so no evaluation took place (dx: 299 parts), and one v2 seed hit the 900 s limit on 222 parts that the other seeds replayed, which may reflect the machine or the programs. After those outcomes were visible we adopted, for all eight runs, a rule giving every setup or resource-limit failure one terminal recovery evaluation with the same judge and limit; programs were not regenerated. It affected 531 records. d0's ten resource failures failed again and stay 0; dx and the v2 seed recovered. On the original records the primary contrast is −0.7951 (dx valid on 1 part) and the three-seed v2 mean is 0.699 instead of 0.930. Both analyses are in Appendix C; we report the recovered one as the measurement and label it as amended.

Runtime. Execution checking costs time: mean minutes per part rise from 7.5 (d0) to 16.0 (dx) and 15.1 (v3); per-run medians from 3.7 to 7.2. These are successful-attempt wall times on CPU nodes that usually ran three parts at once; retried attempts are excluded, so they are not total compute. The trim run's timer starts after its initial replay and repair, so its full latency is not measured.

Pending outputs. Three preregistered analyses are still running: the section-only reference and with it the fourth secondary test ([pending: section-only control (running)]), which tests the trim package against a simple geometric reference; a geometric re-scoring of every delivered tessellation with a different seed and four times the samples ([pending: independent rejudge (running)]), which tests the stability of the IoU estimates; and surface agreement on the fused boundary ([pending: surface agreement (running)]), which tests whether the volumetric gains preserve boundary fidelity. The contrasts in Table 2 are computed from the first scoring pass.

5 Study 2: Frontier agents under the same judge

Harness. An agent receives the target only through tools and edits a model step by step. After every change, the harness assembles the CadQuery program, executes and checks it as in Section 3, and reports build status, validity and executed IoU; the delivered answer is the best executed valid program, or, if no check of a valid program finished within the episode, the current program unverified (it is still re-judged offline like every other delivery). The **code** toolset is `describe_target` (size, volume, flat-face directions and offsets), `section` (exact cross-sections of target and model), `evaluate`, `add_code`, `remove_step`, `list_steps` and `finish`: it gives target inspection and execution feedback. The **CADFit** toolset adds the optimizer's stages: direction detection, sketch extraction on a plane with line/arc fitting, extrusion and revolve fitting, section extrusion, and candidate management. Each episode has 40 tool calls and 30 minutes.

Models and runs. Claude Opus 5.5 (`claude-opus-5-5`) and Sonnet 5.5 (`claude-sonnet-5-5`) run through the Anthropic Messages API with adaptive thinking at effort high, at most 32k output tokens per turn, the full conversation history, and no server-side fallback to another model. Each model runs three independent episodes per part and toolset on

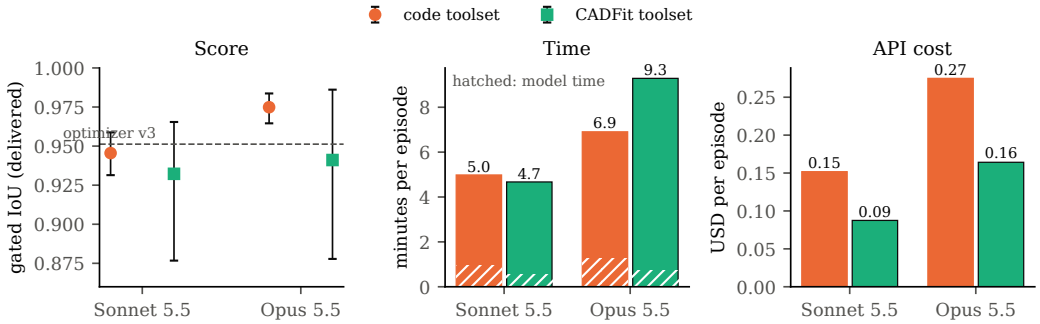


Figure 3: Study 2, 40 reserved fresh2 parts, three episodes per part and condition. Left: gated IoU of the delivered program (mean over parts of the per-part mean; 95% interval); dashed: the optimizer's v3 on the same parts. Middle: mean minutes per episode, hatched part waiting on the model. Right: mean API cost per episode. Runtime and cost are means without intervals.

Table 3: Study 2 against the optimizer on the same 40 parts (gated IoU, 95% interval). Valid: delivered programs that build valid / episodes; no-output episodes ended at the deadline with nothing delivered. Differences are paired over parts (better/worse); exploratory.

	Gated IoU	95% interval	Valid	No output	Mean min	USD / episode
Opus + code	0.975	[0.965, 0.984]	120/120	0	6.9	0.27
Opus + CADFit tools	0.941	[0.878, 0.986]	115/120	5	9.3	0.16
Sonnet + code	0.946	[0.931, 0.959]	120/120	0	5.0	0.15
Sonnet + CADFit tools	0.932	[0.877, 0.965]	117/120	3	4.7	0.09
optimizer d0	0.779	[0.675, 0.876]				
optimizer dx	0.934	[0.902, 0.959]				
optimizer v3	0.951	[0.928, 0.968]				
optimizer + trim	0.958	[0.940, 0.973]				
Opus code − v3	+0.024 [+0.003, +0.048], 25/15					
Opus code − trim	+0.017 [−0.001, +0.036], 22/18					
CADFit tools − code	Opus −0.034 [−0.097, +0.009]; Sonnet −0.013 [−0.068, +0.020]					

the 40 reserved parts; every delivered program is re-judged offline by the same replay and judge as the optimizer. These comparisons were specified after the optimizer study and are exploratory; intervals are nominal.

Results. Opus with the code toolset scores 0.975 with 120/120 valid deliveries (Figure 3, Table 3). On the same parts the optimizer's v3 scores 0.951; the paired difference +0.024 [+0.003, +0.048] excludes zero, while the difference from the optimizer with trim, +0.017 [−0.001, +0.036], does not, so superiority over the full optimizer package is not established. Opus exceeds Sonnet with the code toolset by +0.029 [+0.018, +0.042], better on 37/2 parts.

The optimizer's stages as tools. With the CADFit toolset, the mean differences from code are −0.034 [−0.097, +0.009] for Opus and −0.013 [−0.068, +0.020] for Sonnet. All 472 programs the agents delivered are valid; the CADFit arms also contain 5 (Opus) and 3 (Sonnet) episodes that reached the deadline with nothing delivered, on three parts, and these score zero. Offline validity is not the same as in-episode verification: 22 valid deliveries had not completed an executed check within their episode.

Where the time goes. Tool execution, not the model, dominates episodes: `add_code`, whose cost is the executed check after every change, takes 84–89% of tool time in the code arms, and single calls of the CADFit stages reached 1776 s. Mean minutes per episode are 6.9 (Opus, code) to

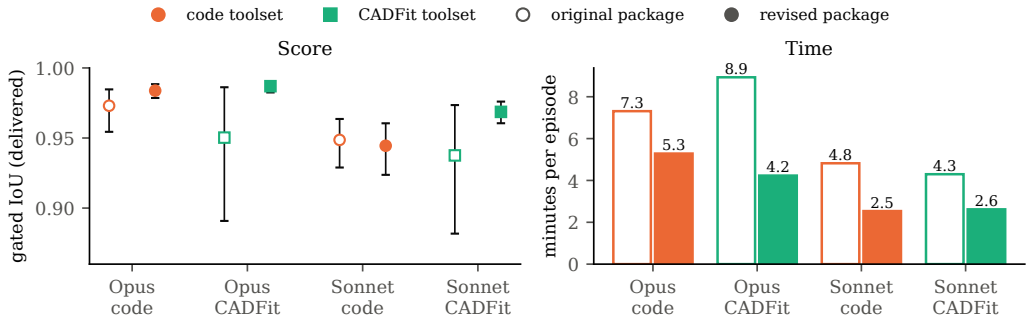


Figure 4: Study 3, 40 fresh3 parts, three episodes per part and condition. Left: gated IoU of the delivered program (mean over parts, 95% interval). Right: mean minutes per episode (means without intervals). Open markers: original package; filled: revised.

9.3 (Opus, CADFit tools); the optimizer's v3 takes 10.4 minutes per part on these parts, on different hardware and without API cost.

6 Study 3: A revised tool package

Profiling. On development parts only, we timed every stage inside the tool calls of a scripted policy that uses the CADFit tools. Sketch extraction spent 99.6% of its time (4 development parts) fitting lines and arcs to every profile on the plane; a function-level profile of one development part attributed most of that to two routines: duplicate-point removal by an all-pairs loop, and a greedy segmenter that refits the whole run with an SVD each time it extends by one point. Revolve fitting rotated points one at a time in Python, and every executed check reloaded and rebuilt the target.

The revised package. Four changes are rewrites intended to preserve outputs (the running-sum form is algebraically equal to the SVD fit; Appendix H), each compared with the original code on development inputs: KD-tree duplicate removal (identical output on 300/300 randomized cases), the segment error from running sums with the old SVD fit deciding whenever a value lies within 10^{-6} (relative) of a threshold it is compared with (identical fitted output on 162/162 sketches from 8 development parts and 9 planes; fitting time 506 s $\rightarrow$ 18 s together with the first change), vectorized rotation (bit-identical on 200/200 cases), and loading the target once per process in the judge (18/18 identical); the checks are recorded in `results/tools/equivalence.json`. One change alters behaviour: each executed check gets at most 120 s. A step whose check does not finish stays in the model unchecked and is never delivered while a checked valid program exists.

Design. The protocol fixed, before any run and before the cohort was drawn: tool package {original, revised} $\times$ toolset {code, CADFit} $\times$ {Opus, Sonnet} on the 40 fresh3 parts, three episodes each (960 episodes), same budget and settings as Study 2, each version running from its own frozen tree, delivered programs re-judged offline. The primary comparison is Opus with CADFit tools, revised $-$ original; the Opus family (primary; CADFit $-$ code under the revised package; code, revised $-$ original) is Holm-corrected over three sign tests; the same comparisons for Sonnet are reported uncorrected. Runtime summaries were prespecified; timing contrasts and their tests are descriptive. Episodes lost to API credit exhaustion, rate limits, a faulty node and one client timeout crash were set aside and rerun before any analysis (Appendix D).

Results. The primary comparison has a mean gain of +0.0367 [+0.0018, +0.0957]; its declared sign test is not significant ($p = 0.121$, Holm 0.243; Table 4). With the revised package, Opus delivers a valid program in all 120/120 episodes with the CADFit tools (original: 116/120, with 1 no-output and 3 invalid), and those tools beat plain code by +0.0032 [+0.0004, +0.0064], better on 28/11 parts, Holm $p = 0.028$. For Sonnet the same contrast is +0.0242 [+0.0097, +0.0447] (uncorrected $p = 0.017$). The best cell, Opus with the revised CADFit tools, scores 0.987 [0.983,

Table 4: Study 3 comparisons (gated IoU, paired over parts; better/worse at $\pm 10^{-9}$; sign test p; Holm over the three Opus tests). Below: mean minutes per part, revised − original (descriptive), with parts faster.

Comparison		Difference & 95% interval	Better/worse	p (Holm)
Opus	CADFit, revised − orig. (primary)	+0.0367 [+0.0018, +0.0957]	22/12	0.121 (0.243)
Opus	CADFit − code, revised	+0.0032 [+0.0004, +0.0064]	28/11	0.009 (0.028)
Opus	code, revised − orig.	+0.0108 [+0.0007, +0.0285]	21/18	0.749 (0.749)
Sonnet	CADFit, revised − orig.	+0.0311 [−0.0014, +0.0837]	18/16	0.864
Sonnet	CADFit − code, revised	+0.0242 [+0.0097, +0.0447]	28/12	0.017
Sonnet	code, revised − orig.	−0.0042 [−0.0088, −0.0002]	15/20	0.500
Minutes, revised − orig.		Opus CADFit −4.7 [−6.6, −2.9]	faster on 33/40	$< 10^{-4}$
		Opus code −2.0 [−3.6, −0.6]	23/40	0.43
		Sonnet CADFit −1.7 [−2.8, −0.7]	25/40	0.15
		Sonnet code −2.3 [−4.0, −0.7]	21/40	0.87

0.991]. Across all 960 episodes, 949 delivered programs are valid offline, of which 48 had not completed an in-episode check.

Time. With the revised package, mean minutes per part with the CADFit tools fall from 8.9 to 4.2 for Opus, faster on 33 of 40 parts, and from 4.3 to 2.6 for Sonnet. In the other cells the mean also falls, but the per-part sign tests are not significant, and the means are dominated by a few parts that used to run long. Opus with code and the original package reproduces Study 2 on this new cohort (0.973 here, 0.975 there). Episodes rerun after infrastructure failures (254 of 960) ran three instead of eight per node, and they are unevenly spread: 75 of 120 for Opus with the original CADFit tools against 23 with the revised ones (all cells in Table 7). Restricted to episodes from the first launch (20 parts with such episodes under both packages), the Opus CADFit-tools time difference is −3.9 [−7.0, −1.0] minutes, faster on 14 of 20 parts (sign test $p = 0.12$): the mean saving remains, while the per-part result above partly rests on rerun episodes.

7 Discussion and limitations

What changes when the program is executed. The largest held-out improvement to the optimizer came from its delivery step rather than from a new proposal family: executing the candidate programs and delivering the best valid one, with fuse-and-repair, recovered most of what the search had found. Under the same execution-based judge, a frontier model with inspection, plain code and execution feedback exceeded the optimizer's v3 on reserved held-out parts, at a few minutes and cents per part. In their original implementation, the optimizer's stages lowered Opus's mean score relative to plain code and lengthened its episodes, while for Sonnet they shortened episodes with no significant score difference; with the revised package they gave Opus a small, consistent edge over plain code. Evaluating tools by their running cost as well as their output therefore changed the conclusion about them.

Limitations. (i) The optimizer study's measurement was amended after outcomes were visible (one terminal recovery evaluation for setup and resource-limit replay failures); the original analysis is reported beside it, and it changes the dx and v2 values substantially. (ii) The agent studies use 40 parts per cohort with three episodes each, so many intervals are several hundredths of IoU wide, and the Study 2 comparisons are exploratory. (iii) One benchmark: hard-tier CADBench parts from two splits. (iv) The endpoint is volumetric IoU; surface agreement and an independent re-judging are pending. (v) The code toolset includes target inspection and execution feedback, so it measures an agent in this harness, not a language model alone. (vi) Closed-model results are tied to specific model versions, settings (32k tokens per turn, full history) and an API; the revised tool package bundles output-preserving rewrites with a behavioural check cap, so its effect is not attributed to either component, and its output equivalence was checked on development inputs only. (vii) Infrastructure failures (faulty nodes, API limits) required reruns, listed in Appendix D, at a different per-node concurrency. (viii) Two fresh3 parts share a document hash with a fresh2 part (Appendix A).

REPRODUCIBILITY STATEMENT

Every result value in the text and tables is a macro generated from committed result files by `bench/paper_numbers.py`, and every figure is drawn by a committed script; configuration constants, cohort dates and the counts of the development example in Figure 2 come from the protocols, the code and the pipeline's records. The protocols, with their dated amendments and execution logs, the frozen source trees of every arm, the cohort definitions (seeded draws excluding earlier model indices), the scoring and analysis code, and the per-part and per-episode records will be released with the paper. Closed-model episodes require an Anthropic API key; their raw records are included, so the analyses can be re-run without the API.

REFERENCES

Jesse Barkley, Rumi Loghmani, and Amir Barati Farimani. CADSmith: Multi-agent CAD generation with programmatic geometric validation, 2026.

CadQuery contributors. CadQuery: A python parametric CAD scripting framework based on OCCT. https://github.com/CadQuery/cadquery, 2024.

Anna C. Doris, Jacob Thomas Sony, Ghadi Nehme, Era Syla, Amin Heyrani Nobari, and Faez Ahmed. Cadbench: A multimodal benchmark for ai-assisted cad program generation, 2026. URL https://arxiv.org/abs/2605.10873.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. URL https://www.jstor.org/stable/4615733.

Tao Hu, Jiaxin Ai, Licheng Wen, Xueheng Li, Shu Zou, Siqi Li, Nianchen Deng, Xinyu Cai, Hongbin Zhou, Pinlong Cai, Daocheng Fu, Yu Yang, Hairong Zhang, Botian Shi, and Xuemeng Yang. Itercad: An iterative multimodal agent for visually-grounded cad generation and editing, 2026. URL https://arxiv.org/abs/2606.13368.

Alec Jacobson, Ladislav Kavan, and Olga Sorkine-Hornung. Robust inside-outside segmentation using generalized winding numbers. *ACM Transactions on Graphics*, 32(4), 2013. doi: 10.1145/2461912.2461916.

Soslan Kabisov, Vsevolod Kirichuk, Andrey Volkov, Gennadii Savrasov, Marina Barannikov, Anton Konushin, Andrey Kuznetsov, and Dmitrii Zhemchuzhnikov. Cadreasoner: Iterative program editing for cad reverse engineering, 2026. URL https://arxiv.org/abs/2603.29847.

Sebastian Koch, Albert Matveev, Zhongshi Jiang, Francis Williams, Alexey Artemov, Evgeny Burnaev, Marc Alexa, Denis Zorin, and Daniele Panozzo. Abc: A big cad model dataset for geometric deep learning, 2019. URL https://arxiv.org/abs/1812.06216.

Emmett Lambert and Manifold contributors. Manifold: geometry library for topological robustness. https://github.com/elalish/manifold, 2024.

Dimitrios Mallis, Ahmet Serdar Karadeniz, Sebastian Cavada, Danila Rukhovich, Niki Foteinopoulou, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-assistant: Tool-augmented vllms as generic cad task solvers, 2025. URL https://arxiv.org/abs/2412.13810.

Ghadi Nehme, Eamon Whalen, and Faez Ahmed. Cadfit: Precise mesh-to-cad program generation with hybrid optimization, 2026. URL https://arxiv.org/abs/2605.01171.

Danila Rukhovich, Elona Dupont, Dimitrios Mallis, Kseniya Cherenkova, Anis Kacem, and Djamila Aouada. Cad-recode: Reverse engineering cad code from point clouds, 2025. URL https://arxiv.org/abs/2412.14042.

Gennadiy Savrasov, Maksim Elistratov, Nikita Gavrilov, Albert Garifullin, Oleg Pavlov, Soslan Kabisov, Vladimir Frolov, Anton Konushin, Andrey Kuznetsov, and Dmitrii Zhemchuzhnikov. Cadfather: Autonomous cad reconstruction through coordinated tool use, 2026. URL https://arxiv.org/abs/2610.09127.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL https://arxiv.org/abs/2302.04761.

Mikaela Angelina Uy, Yen yu Chang, Minhyuk Sung, Purvi Goel, Joseph Lambourne, Tolga Birdal, and Leonidas Guibas. Point2cyl: Reverse engineering 3d objects from point clouds to extrusion cylinders, 2022. URL https://arxiv.org/abs/2112.09329.

Karl D. D. Willis, Yewen Pu, Jieliang Luo, Hang Chu, Tao Du, Joseph G. Lambourne, Armando Solar-Lezama, and Wojciech Matusik. Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences, 2021. URL https://arxiv.org/abs/2010.02392.

Rundi Wu, Chang Xiao, and Changxi Zheng. Deepcad: A deep generative network for computer-aided design models, 2021. URL https://arxiv.org/abs/2105.09492.

Xianghao Xu, Wenzhe Peng, Chin-Yi Cheng, Karl D. D. Willis, and Daniel Ritchie. Inferring cad modeling sequences using zone graphs, 2021. URL https://arxiv.org/abs/2104.03900.

Shuhao Yan, Changhao He, Peng Hu, and Xi Peng. RA-CAD: Learning post-execution critique for state-aware text-to-CAD generation, 2026.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL https://arxiv.org/abs/2405.15793.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL https://arxiv.org/abs/2210.03629.

A COHORTS

All cohorts are seeded draws from the CADBench index (hard tier) by `bench/fetch_cadbench.py`, taken in a fixed order so that adding a cohort never changes an earlier one, and excluding any part whose model index (the first field of its file id) appears in an earlier set. The exclusion is by model index only: two fresh3 parts share the second file-id field (a document hash) with a fresh2 part (00078986 with 00078985, 00894209 with 00894237), so near-relatives of two fresh2 parts may appear in fresh3. *beamfix5* is five parts kept in `bench/cadbench/beamfix5`. Development used *test* (100 benchA), *dev* (30 + 30), *smoke* (5), *beamfix5* (5) and *fresh* (30 + 30). *fresh2* (150 + 150) was drawn on 2026-10-18 after the optimizer repairs were developed. The 40 reserved parts were drawn on 2026-11-01 from fresh2's manifest alone (seed 20261101, 20 per split) before any fresh2 outcome. *fresh3* (20 + 20) was drawn on 2026-11-06 after the tools-v3 code and protocol were committed; it shares no model index with any earlier set.

B FAILURE CLASSES

For the agent studies, delivered-valid counts and no-output episodes are in Table 3; in Study 3 the only invalid deliveries were 3 (Opus, CADFit tools v2), 1 (Opus, code v2), 1 and 1 (Sonnet, code v2 and v3) and 1 (Sonnet, CADFit tools v2); no-output episodes: 1 (Opus, CADFit v2) and 3 (Sonnet, CADFit v2), none with tools v3.

Table 5: Study 1 delivery outcome per part and run (300 parts each), after the recovery amendment and, in brackets, on the original records where they differ.

Run	Valid	Built invalid	Build error	Missing	Resource limit	Sandbox setup
d0	259	23	7	1	10	0
dx	293 [1]	1 [0]	0	0	0	6 [299]
v2 seed 0	299	1	0	0	0	0
v2 seed 1	298	2	0	0	0	0
v2 seed 2	298 [77]	2 [1]	0	0	0 [222]	0
v3 seed 0	300	0	0	0	0	0
v3 seed 1	300	0	0	0	0	0
v3 seed 2	298	1	0	1	0	0

C Recovery amendment: original and recovered analyses

Eligibility (decided 2026-11-05, after the affected score files existed, applied to all eight runs): a replay failure whose cause is a sandbox that could not be set up (*setup*; no evaluation took place) or an rc -9 kill or the 900 s limit (*resource*; machine or program). Each eligible part received one recovery evaluation with the same judge, limit and the scorer's existing single retry of a silent replay death, at fixed concurrency; the result is terminal, and a repeated failure scores 0. Programs were not regenerated and no search or selection was redone. Affected records: 299 dx (setup), 222 v2 seed 2 (resource), 10 d0 (resource); total 531.

Table 6: Gated IoU on the original records and after recovery (300 parts).

	Original	Recovered
d0	0.798	0.798
dx	0.002	0.901
v2 seed 2	0.238	0.930
v2, three-seed mean	0.699	0.930
dx − d0 (primary)	–0.7951	+0.1037

D Exclusions, reruns and cost

Events not visible in the result files are recorded in the execution logs of the two protocols (`review-stage/HELDOUT_PROTOCOL.md`, `review-stage/TOOLS_V3_PROTOCOL.md`). **Study 1.** Pipeline chunks that failed for infrastructure reasons (nodes without the software stack, out-of-memory workers) were resubmitted under the same frozen trees as recorded in the protocol's execution log; the first dx recovery job was scheduled on a node whose sandboxed replays timed out elsewhere and was cancelled before writing results, then resubmitted excluding that node. **Study 2.** A first Opus run on one node was set aside: 43 of its 44 diagnosed episodes contain executed-call timeouts at 90 s (578 in total); 35 episodes wrote records, with $8.95 recorded cost (the cost of the rest is unknown). It was rerun on other nodes, and every later job first replays a box through the sandboxed path and stops before any API call if that fails. Retained Study 2 episodes cost $81.40. **Study 3.** 239 episodes ended on API errors (credit exhaustion and rate limits; $7.44 recorded) and were moved aside and rerun; one job stopped at its preflight on a faulty node (53 episodes not started) and one episode crashed at its deadline when the frozen client did not catch a streaming read timeout (no record written); both were rerun from the same frozen tree. Retained Study 3 episodes cost $154.96.

E Harness and settings

Agents see the target only through tools; frames are normalized (largest extent spans $[-1, 1]$). Tool replies report the executed program's build status, validity and IoU after every change; the delivered answer is the best executed valid program (valid first, then IoU). The delivered answer is the

best executed valid program; if no check of a valid program finished within the episode, the current program is exported unverified and re-judged offline like every delivery. Claude runs use the official SDK (configuration in `agent/anthropic_chat.py` and `bench/claude_job.sh`) with streaming, adaptive thinking, effort high, a 32k-token limit per turn, prompt caching, the full native message history (tool results are not truncated), and no server-side fallback; a refusal or API error ends an episode. Episodes run 3–8 at a time per CPU node (the `-jobs` setting of each submission). In Study 3 the two tool versions are identified by the frozen source tree each job ran from (output paths under `tv_v2_src` and `tv_v3_src`); the episode records' `tools` field names the harness mode, which is the same for both.

F JUDGE

Normalization. The target mesh is scaled and centred so that its largest bounding-box extent spans $[-1, 1]$; the same transform is applied to the reconstruction. **Exact path.** If the target is watertight and the reconstruction is watertight, consistently wound and of positive volume, each closed body of the reconstruction is converted to a manifold, overlapping bodies are unioned and inward-facing bodies (cavities) subtracted, and IoU $= |A \cap B|/|A \cup B|$ is computed from manifold boolean volumes of the two polyhedra; the result is accepted only if the boolean volumes are mutually consistent. **Monte-Carlo path.** Otherwise, points are drawn uniformly in the union of the two bounding boxes padded by 0.01, in batches of 40,000 with a fixed seed, until the union contains at least 10,000 points or five batches have been drawn. A point is inside the target by ray casting (watertight target) or generalized winding number > 0.5, and inside the reconstruction if its generalized winding number exceeds 0.5. IoU is the ratio of intersection to union counts. **Executed program.** The CadQuery result (`final_result` or `result`) must be built; OCC `isValid` must hold for the shape and for every solid; the shape is tessellated with linear tolerance 10^{-3} and angular tolerance 0.1.

G ARMS OF STUDY 1

All arms run `run_pipeline.py` from frozen source trees with these flags. Common to all: `-persistent-cq -exact-union -cut-phase -event-slices -detect-directions -translation-max-points 400 -beam 4 -section-extrude`. d0 runs them from the pre-repair snapshot (commit `e260397`) with the two infrastructure patches; dx adds `-exec-verify`; v2 adds `-section-original -score-original -shared-fixes`; v3 adds `-precise-extents`; trim runs the execution check's trim stage on each v3 run's delivered program. **Delivery with `-exec-verify`.** (1) Execute the search's program and a rewrite that replaces stacked compounds by boolean fuse and cut; (2) judge every executed shape that builds; (3) deliver the valid shape with the highest IoU; (4) if none is valid, repair each invalid solid with ShapeFix and drop solids that stay invalid, then return to (3); (5) otherwise deliver the search's program unchanged (it then scores 0).

H OUTPUT-PRESERVING REWRITES

For a run of points $x_a, \ldots, x_b \in \mathbb{R}^2$ with $m = b-a+1$, let $S = \sum x_i$ and $Q = \sum x_i x_i^\top$ over the run. The centred scatter matrix is $C = Q - SS^\top/m$, and the root-mean-square perpendicular distance of the best-fit line equals $\sqrt{\lambda_{\min}(C)/m}$, where $\lambda_{\min}(C) = \frac{1}{2}(C_{11}+C_{22}) - \sqrt{\frac{1}{4}(C_{11}-C_{22})^2 + C_{12}^2}$. In exact arithmetic this equals the original code's value: with $\sigma_{\min}$ the smallest singular value of the centred points, $\lambda_{\min}(C) = \sigma_{\min}^2$ and the RMS distance is $\sigma_{\min}/\sqrt{m}$. Prefix sums make it $O(1)$ per run. Only this error enters the segmenter's decisions for lines (a line's output is the run's end points). In floating point the two computations can differ (the prefix-sum form subtracts large terms); as a safeguard, whenever the fast value lies within a relative 10^{-6} of the threshold or of the competing circle error, the original SVD computation decides. We do not have an error bound guaranteeing that differences stay inside that band; equivalence beyond the exact-arithmetic identity rests on the tests reported here. Duplicate removal keeps a point unless it lies within the tolerance of an earlier kept point; the KD-tree version tests only candidate neighbours found within a slightly larger radius and applies the original distance test to them. Equivalence counts for all four rewrites are in `results/tools/equivalence.json`.

I STUDY 3: ALL CELLS

Table 7: Study 3, 40 fresh3 parts, three episodes per part and cell. Gated IoU (95% interval); valid deliveries / episodes; no-output episodes; mean and median minutes per episode; API cost of the cell; episodes rerun after infrastructure failures (three per node instead of eight).

Model	Toolset, package	Gated IoU	95% interval	Valid	No out.	Mean min	Median min	Reruns
Opus	code, original	0.973	[0.954, 0.985]	119/120	0	7.3	2.6	75
Opus	code, revised	0.984	[0.979, 0.988]	120/120	0	5.3	2.6	22
Opus	CADFit, original	0.950	[0.891, 0.986]	116/120	1	8.9	3.5	75
Opus	CADFit, revised	0.987	[0.983, 0.991]	120/120	0	4.2	2.0	23
Sonnet	code, original	0.949	[0.929, 0.964]	119/120	0	4.8	1.5	18
Sonnet	code, revised	0.944	[0.924, 0.961]	119/120	0	2.5	1.6	13
Sonnet	CADFit, original	0.938	[0.882, 0.974]	116/120	3	4.3	1.8	18
Sonnet	CADFit, revised	0.969	[0.961, 0.976]	120/120	0	2.6	1.2	10

J AGENT TOOLS

Code toolset: `describe_target` (size, volume, flat-face directions and offsets), `section` (exact cross-section polygons of target and current model on an axis-aligned plane), `evaluate` (current executed IoU and where material is missing or extra), `add_code` (a CadQuery snippet defining `result`, added or cut), `remove_step`, `list_steps`, `finish`. **CADFit toolset** adds `detect_directions`, `extract_sketch` (slice on a plane, nest loops into profiles, fit lines and arcs), `fit_extrude` and `fit_revolve` (candidates from a sketch with their IoU), `section_extrude`, `list_candidates`, `show_candidate` and `add_candidate`. The system prompts and the JSON tool schemas are in `agent/agent_loop.py` and `agent/cad_tools.py` of the supplementary code.